Université de Lille - BUT SD - Semestre 4 - 2025/2026

SAE 4.VCOD01

Intégration de données dans un Data Warehouse

Migration Oracle vers SQL Server, ajout d'une source Magasin
et implémentation du SCD Type 2 avec SSIS

Par

Kris LOKOUN et Abel THYS

Sous la direction de

M. LEVELER

Date de dépôt : Mars 2026

Table des matières

I. Introduction **3**

II. Méthodologie **4**

1. Migration technologique 4

2. Construction des couches de données 4

3. Création des flux ETL avec SSIS 4

4. Validation et optimisation 5

III. Détail du projet **6**

1. Description du projet 6

A. Contexte et continuité du S3 6

B. Organisation du travail 6

C. Outils et technologies utilisés 6

2. Analyse initiale et modélisation 8

3. Architecture en 3 couches (DSA, ODS, DWH) 11

A. Data Staging Area (DSA) 11

B. Operational Data Store (ODS) 11

C. Data Warehouse (DWH) et modèle en étoile 12

4. Création des flux de données avec SSIS 15

A. Flux Sources vers DSA 15

B. Flux DSA vers ODS 16

C. Flux ODS vers DWH 17

5. Gestion du SCD Type 2 sur DIM_TRACK 21

6. Validation et vérification des données 25

IV. Automatisation de l'ETL **28**

V. Difficultés rencontrées et solutions **29**

VI. Conclusion **30**

VII. Bilan personnel **31**

VIII. Remerciements **32**

I. Introduction

Dans le cadre de la SAE 4. VCOD (Système d'Information Décisionnel), nous poursuivons le travail initié au semestre 3 (SAE 3.02) en intégrant des données dans un entrepôt de données (Data Warehouse). Ce projet constitue la suite directe de notre travail précédent, avec des évolutions majeures tant sur le plan technologique que fonctionnel.

Au semestre 3, nous avions construit une chaîne ELT (Extraction, Loading, Transformation) à partir de la seule base de données Chinook, en utilisant Oracle Database et Oracle Data Integrator (ODI). Chinook est une base de données transactionnelle modélisant un magasin de musique en ligne, comparable à iTunes. Elle comporte 11 tables représentant les différentes entités du système : les artistes (Artist), les albums (Album), les pistes musicales (Track), les genres (Genre), les types de média (MediaType), les playlists (Playlist, PlaylistTrack), les clients (Customer), les employés (Employee), ainsi que les factures (Invoice) et leurs détails (InvoiceLine). Au total, la base contient environ 15 000 lignes de données réparties entre ces tables.

L'architecture en trois couches (DSA, ODS, DWH) avait été mise en place avec un modèle décisionnel en étoile (star schema) dans le Data Warehouse. Cependant, certains aspects n'avaient pas pu être finalisés, notamment l'implémentation effective du SCD Type 2 (Slowly Changing Dimension) via l'IKM Oracle Slowly Changing Dimension, qui n'avait pas fonctionné comme attendu.

Au semestre 4, le projet évolue sur trois axes majeurs :

- Migration technologique : passage de l'écosystème Oracle (Oracle Database + ODI) versl'écosystème Microsoft (SQL Server 2022 + SSIS). Ce changement nous permet de découvrir un autre ensemble d'outils largement utilisé dans l'industrie de la Business Intelligence. SQL Server Integration Services (SSIS) est l'outil ETL intégré à la suite SQL Server, permettant de créer des packages de flux de données visuellement dans Visual Studio.
- Ajout d'une seconde source de données : la base Magasin, représentant les ventes en magasinphysique. Cette base, initialement fournie en syntaxe Oracle (PL/SQL), contient 2 tables (Invoice et InvoiceLine) représentant environ 2 000 lignes de transactions. Le Data Warehouse doit désormais fusionner les deux canaux de vente (Online via Chinook et Store via Magasin) tout en maintenant la traçabilité de l'origine des données grâce à un identifiant de canal (CHANNEL_ID).
- Implémentation du SCD Type 2 : gestion de l'historique des changements de prix dans la dimensionDIM_TRACK. Contrairement au S3 où nous avions tenté d'utiliser le mécanisme intégré d'ODI, nous implémentons ici le SCD2 manuellement avec les composants natifs de SSIS (Lookup, Conditional Split, OLE DB Command, Derived Column). Cette approche manuelle nous donne un contrôle total sur le processus d'historisation.

Ce rapport présente en détail les étapes de conception et de réalisation : la méthodologie adoptée, l'analyse de la base source, la construction des couches de données, la création des flux SSIS, l'implémentation du SCD Type 2, la validation des résultats, ainsi que les difficultés rencontrées et les solutions apportées.

II. Méthodologie

Le projet suit une méthodologie structurée en quatre étapes principales, chacune correspondant à une phase distincte du cycle de vie d'un projet d'intégration de données. Cette méthodologie est identique à celle du S3, avec des adaptations liées au changement d'outils.

1. Migration technologique

La première étape consiste à migrer l'ensemble du projet de l'écosystème Oracle vers l'écosystème Microsoft. Cela implique plusieurs sous-étapes :

- Installation et configuration de SQL Server 2022 sur les postes de travail, avec création des 7 bases de données nécessaires (Chinook, Magasin, DSA_ONLINE, DSA_STORE, ODS_ONLINE, ODS_STORE, DWH_COMMON).
- Conversion des scripts SQL Oracle (PL/SQL) vers T-SQL (Transact-SQL) : les types de données différent (NUMBER -> INT/DECIMAL, VARCHAR2 -> VARCHAR, DATE -> DATETIME), les fonctions également (TO_DATE -> CAST, NVL -> ISNULL, SYSDATE -> GETDATE ()), et les séquences Oracle sont remplacées par des colonnes IDENTITY.
- Remplacement d'ODI par SSIS pour la création des flux ETL. Les interfaces et procédures ODI sont reconstituées sous forme de packages SSIS dans Visual Studio 2022, avec l'extension SQL Server Data Tools (SSDT).

2. Construction des couches de données

L'architecture est construite en trois couches, conformément aux bonnes pratiques de l'entreposage de données. Par rapport au S3, la principale évolution est le dédoublement des couches DSA et ODS pour gérer les deux sources séparément avant la fusion dans le DWH :

DSA (Data Staging Area) : deux bases séparées (DSA_ONLINE et DSA_STORE). Zone de transit vidée (TRUNCATE) à chaque chargement.

Ajout d'un CHANNEL_ID (1 = Online, 2 = Store) via un composant Derived Column pour identifier l'origine des données dès l'entrée dans le système.

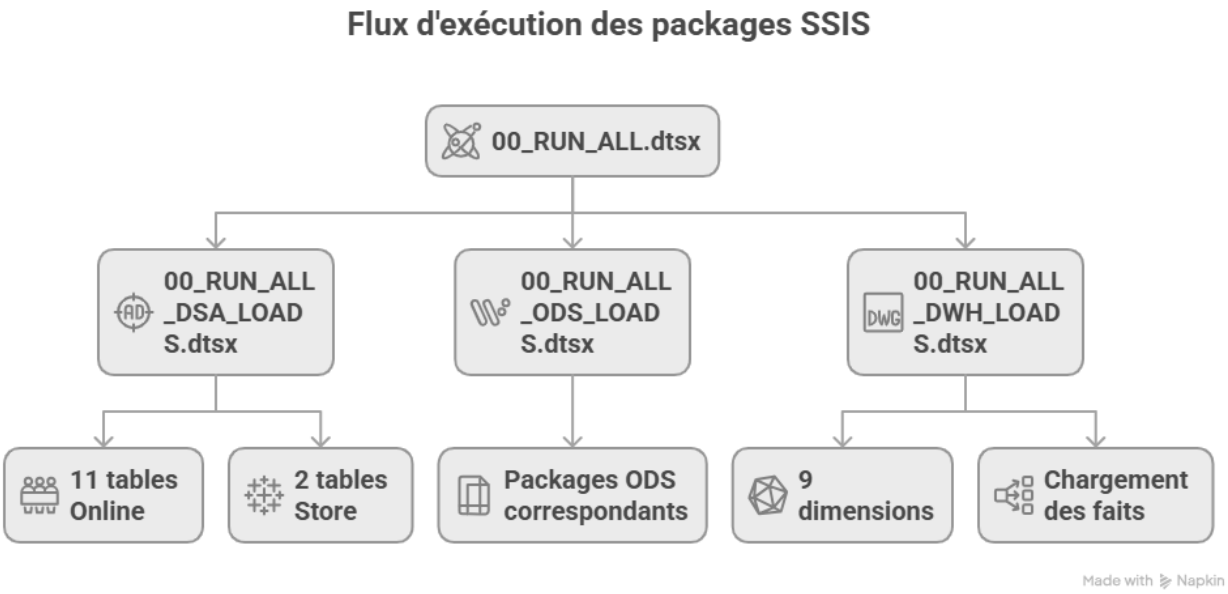
ODS (Operational Data Store) : deux bases avec ajout d'une colonne LOAD_DATE pour la traçabilité temporelle.

L'ODS n'est PAS vidé à chaque chargement : un Lookup anti-doublon évite les insertions en double, et une OLE DB Command met à jour les lignes existantes si elles ont changé.

DWH (Data Warehouse) : base unique (DWH_COMMON) fusionnant les deux sources via Union All. Modèle en étoile avec 9 dimensions (dont DIM_TRACK en SCD2) et 2 tables de faits (FACT_INVOICE et FACT_INVOICE_LINE).

3. Création des flux ETL avec SSIS

Les flux ETL sont implémentés sous forme de packages SSIS (.dtsx) dans Visual Studio 2022. Chaque package contient un Control Flow (flux de contrôle) définissant l'ordre des tâches, et un ou plusieurs Data Flow Tasks (flux de données) définissant les transformations. Plus de 30 packages ont été créés, organisés en trois niveaux d'orchestration :



Cette structure hiérarchique permet de réexécuter l'ensemble de l'ETL en un seul clic, tout en conservant la possibilité de tester chaque package individuellement.

4. Validation et optimisation

À chaque étape de l'intégration, nous avons mis en place des contrôles rigoureux pour valider la qualité des données :

- Comptage des lignes par couche (Source -> DSA -> ODS -> DWH) pour vérifier la cohérence et détecter les pertes de données.
- Calcul du chiffre d'affaires (CA) à chaque couche pour s'assurer de la préservation des valeurs monétaires.
- Test du SCD Type 2 : modification volontaire de 48 prix dans la source Chinook, puis vérification de l'apparition de deux versions dans DIM_TRACK (ancienne fermée, nouvelle active).
- Composant Row Count dans chaque package DSA pour compter les lignes transférées et stocker le résultat dans une variable SSIS.

III. Détail du projet

1. Description du projet

A. Contexte et continuité du semestre 3

Ce projet s'inscrit dans la continuité directe de la SAE 3.02 réalisée au semestre 3. Au S3, nous avions construit un Data Warehouse à partir de la seule base Chinook en utilisant Oracle Database comme SGBD et Oracle Data Integrator (ODI) comme outil ETL. Le modèle en étoile comprenait 8 dimensions et 2 tables de faits, alimentées par des interfaces ODI.

Cependant, le semestre 3 avait révélé certaines limitations : l'installation d'ODI était complexe et ne fonctionnait qu'à l'IUT, nous empêchant de travailler à domicile. De plus, le SCD Type 2 configuré via l'IKM Oracle n'avait pas produit les résultats attendus. Ces difficultés ont motivé le changement de technologie au S4.

Au semestre 4, nous conservons la même architecture logique (DSA -> ODS -> DWH) tout en apportant trois évolutions majeures : le changement de technologie (SQL Server + SSIS), l'ajout d'une seconde source de données (Magasin), et l'implémentation effective du SCD Type 2 avec un contrôle total sur le processus.

B. Organisation du travail

Le projet a été réalisé en binôme (Kris LOKOUN et Abel THYS) sur une période de novembre 2025 à mars 2026, représentant environ 80 heures de travail. Nous avons travaillé à la fois à l'IUT et à domicile, grâce à la facilité d'installation de SQL Server et Visual Studio sur nos machines personnelles (contrairement à ODI au S3).

La répartition des tâches a été la suivante :

Kris LOKOUN : conception et création des packages SSIS (30+ packages), implémentation du SCD Type 2 (9 composants), débogage des flux, vérification de la cohérence des données, rédaction du rapport et de l'exposé

Abel THYS : rédaction des scripts SQL de création des bases (7 scripts), conversion de la base Magasin d'Oracle vers SQL Server, tests d'intégration et débogage, vérification des requêtes de validation

Cette répartition a permis d'avancer en parallèle sur les aspects techniques (scripts SQL) et visuels (packages SSIS), tout en maintenant une communication régulière pour assurer la cohérence globale du projet.

C. Outils et technologies utilisées

Le projet s'appuie entièrement sur l'écosystème Microsoft, contrairement au S3 qui utilisait l'écosystème Oracle. Ce choix a été imposé par les consignes de la SAE, mais il nous permet de découvrir un autre ensemble d'outils majeur du marché de la BI. Voici les outils utilisés et leur rôle précis :

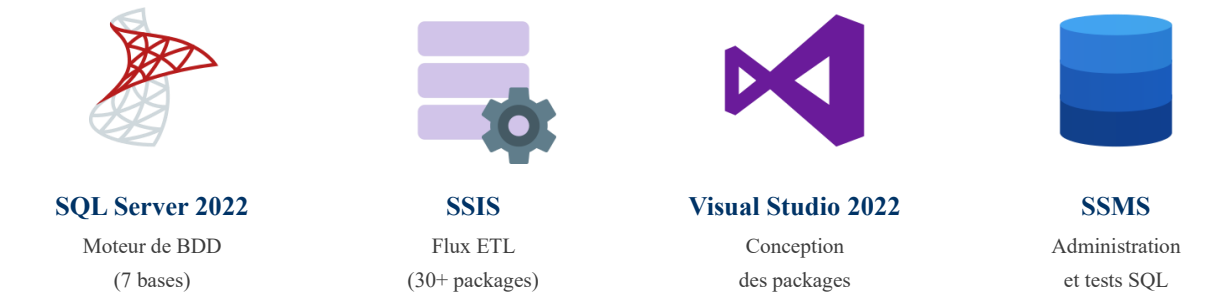


Figure 1 : Outils et technologies utilisés

SQL Server 2022 est le moteur de base de données relationnelle qui héberge nos 7 bases. Il gère les connexions, les transactions et l'exécution des requêtes T-SQL. L'avantage par rapport à Oracle est la simplicité d'installation et la gratuité de la version Developer.

SSIS (SQL Server Integration Services) est l'outil ETL qui remplace ODI. Il permet de créer des packages visuels dans Visual Studio, avec deux niveaux de conception : le Control Flow (flux de contrôle, qui définit l'ordre des tâches) et le Data Flow (flux de données, qui définit les transformations ligne par ligne). Chaque composant est configurable via un éditeur graphique.

SSMS (SQL Server Management Studio) est l'outil d'administration qui permet d'écrire et tester les requêtes SQL, de visualiser les données et de vérifier les résultats des chargements. C'est l'équivalent d'Oracle SQL Developer utilisé au S3.

Outil	Equivalent S3	Rôle dans le projet
SQL Server 2022	Oracle Database	7 bases de données
SSIS	ODI	30+ packages ETL
Visual Studio 2022	ODI Studio	Conception SSIS
SSMS	SQL Developer	Administration + tests
T-SQL	PL/SQL	MERGE, CTE, triggers
SQL Server Agent	-	Planification automatique

2. Analyse initiale et modélisation

Avant de construire les couches de données, il est essentiel d'analyser les bases sources pour en comprendre la structure, les relations et les volumes. Cette analyse permet de déterminer quelles tables et colonnes seront utiles dans le Data Warehouse, et comment les transformer en modèle décisionnel.

Terminologie de la modélisation

La modélisation des données passe par trois niveaux d'abstraction, du plus conceptuel au plus technique :

MCD (Modèle Conceptuel) : représentation abstraite des entités et de leurs relations, indépendante de tout SGBD.

Par exemple : un Artiste compose des Albums, un Album contient des Pistes.

MLD (Modèle Logique) : traduction du MCD en tables avec des colonnes nommées, des clés primaires (PK) et des clés étrangères (FK).

Les relations sont matérialisées par des tables d'association

(Ex : PlaylistTrack).

MPD (Modèle Physique) : le MLD avec les types de données spécifiques au SGBD (INT, NVARCHAR, DECIMAL, DATETIME pour SQL Server). C'est le schéma directement implémentable.

Analyse de la base Chinook

La base Chinook est un modèle transactionnel (OLTP) classique comportant 11 tables organisées en deux zones fonctionnelles :

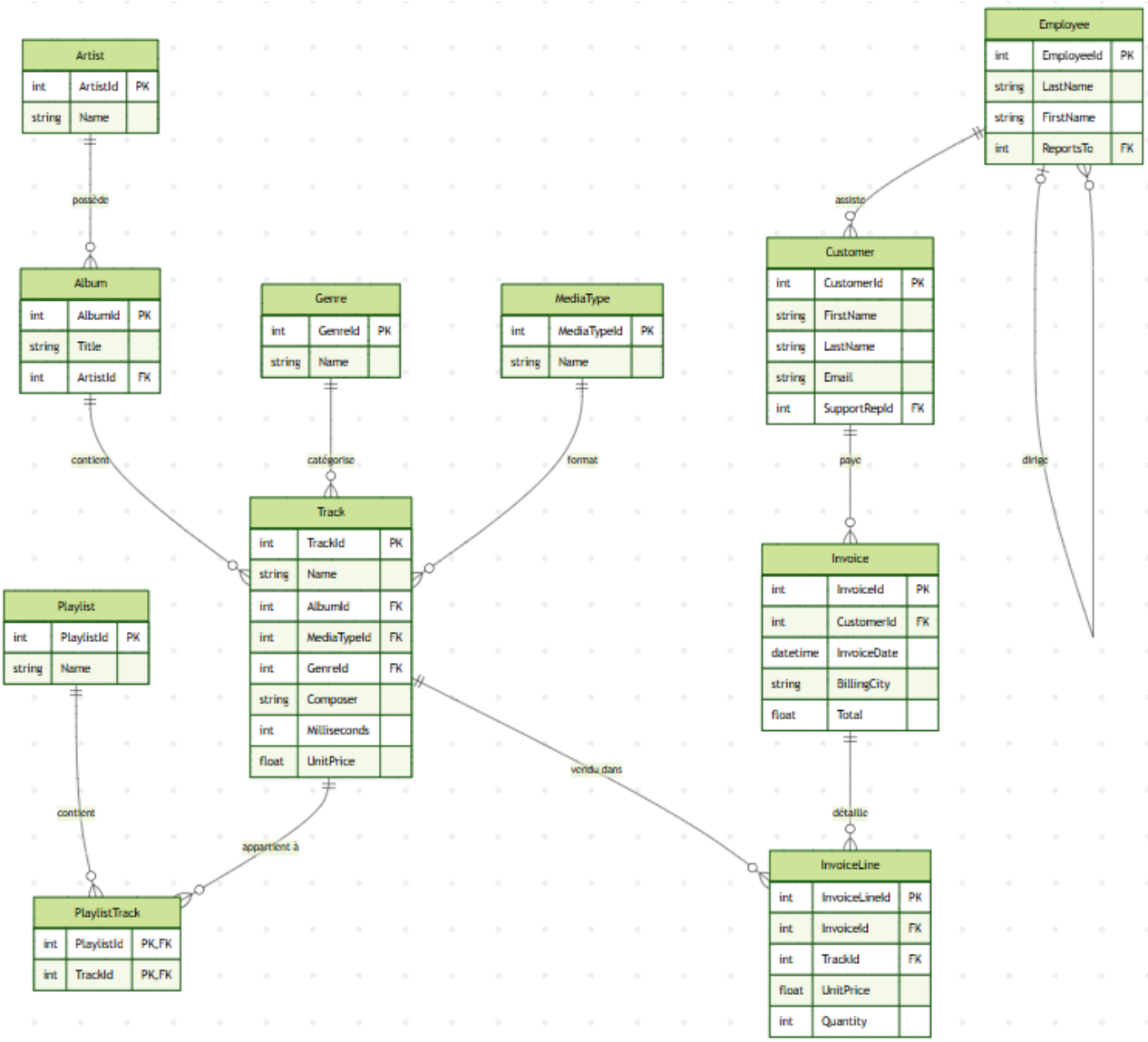
Comparaison des zones Musique et Ventes

Caractéristique	Zone Musique	Zone Ventes
Nombre de tables	7	4
Table centrale	Track	InvoiceLine
Relations	AlbumId, GenreId, MediaTypeId	TrackId, InvoiceId
SCD Type 2	Track	N/A
Auto-référence	N/A	Employee (ReportsTo)

Made with Napkin

Le Modèle Physique de Données (MPD) ci-dessous représente les 11 tables avec leurs clés primaires (PK) et étrangères (FK). Les traits représentent les relations entre tables. Ce schéma est le point de départ pour concevoir notre Data Warehouse.

MPD de la base Chinook (11 tables)



PK = Clé primaire | FK = Clé étrangère | Traits = Relations entre tables

Figure 2 : MPD de la base Chinook (11 tables, SQL Server)

Dictionnaire de données (tables principales)

Table	Colonnes principales	Lignes	Rôle
Artist	ArtistId (PK), Name	275	Artistes musicaux
Album	AlbumId (PK), Title, ArtistId (FK)	347	Albums
Track	TrackId (PK), Name, UnitPrice, ...	3503	Pistes (SCD2)
Genre	GenreId (PK), Name	25	Genres musicaux
MediaType	MediaTypeId (PK), Name	5	Types de support
Customer	CustomerId (PK), FirstName, ...	59	Clients
Employee	EmployeeId (PK), ReportsTo (FK)	8	Employés
Invoice	InvoiceId (PK), CustomerId (FK), ...	412	Factures
InvoiceLine	InvoiceLineId (PK), InvoiceId, ...	2240	Détails

Analyse de la base Magasin

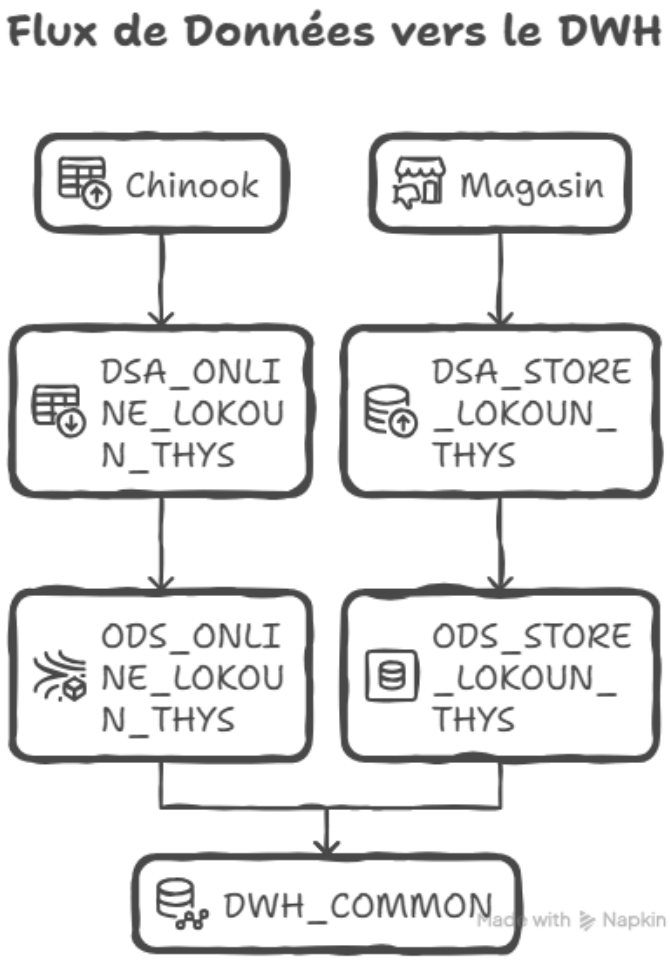
La base Magasin est beaucoup plus simple : elle ne contient que 2 tables (Invoice et InvoiceLine) représentant les ventes en magasin physique, pour environ 2 000 lignes. Les clés étrangères vers Track (TrackId) permettent de croiser les données avec Chinook dans le DWH. Cette base était initialement fournie en syntaxe Oracle (PL/SQL) avec des types comme NUMBER et VARCHAR2, nécessitant une conversion manuelle vers T-SQL.

Du modèle OLTP au modèle OLAP

Le passage du modèle transactionnel (OLTP) au modèle décisionnel (OLAP) nécessite une transformation fondamentale : les tables normalisées de Chinook sont dénormalisées en dimensions et en faits. Les clés naturelles (NK : ArtistId, TrackId, ...) sont remplacées par des clés techniques (TK : TK_ARTISTE, TK_PISTE, ...) générées automatiquement via des colonnes IDENTITY. Cette transformation est la clé de voûte du Data Warehouse.

3. Architecture en 3 couches (DSA, ODS, DWH)

L'architecture ETL repose sur trois couches de données, chacune ayant un rôle précis dans le processus d'intégration. Cette architecture en couches est un standard de l'industrie car elle permet de séparer les responsabilités : la DSA gère l'extraction brute, l'ODS gère la traçabilité, et le DWH gère l'analyse. Par rapport au S3, la principale évolution est le dédoublement des couches DSA et ODS pour gérer les deux sources séparément :



A. Data Staging Area (DSA)

La DSA est la zone de transit entre les sources et le reste du système. Son rôle est de copier les données brutes des sources sans aucune transformation métier. Deux bases séparées ont été créées : DSA_ONLINE_LOKOUN_THYS (11 tables, miroir de Chinook) et DSA_STORE_LOKOUN_THYS (2 tables, miroir de Magasin).

Caractéristiques techniques de la DSA :

- TRUNCATE à chaque chargement : la DSA est vidée intégralement avant chaque transfert, via un Exécute SQL Task au début de chaque package. Cela garantit que seules les données les plus récentes de la source sont présentes. C'est une approche Full Load (rechargement complet).
- Ajout de colonnes techniques via Derived Column : une date de chargement (ex : TrackDate = GETDATE ()) et un identifiant de canal (CHANNEL_ID = 1 pour Online, 2 pour Store). Ces colonnes n'existent pas dans les sources et sont calculées à la volée lors du transfert.
- Aucune transformation métier : les données sont copiées telles quelles. Les noms de colonnes et lestypes restent identiques à la source (sauf les ajouts techniques).

B. Operational Data Store (ODS)

L'ODS est la couche intermédiaire entre la DSA et le DWH. Son rôle principal est d'assurer la traçabilité temporelle des données : contrairement à la DSA, l'ODS n'est PAS vidé à chaque chargement. Il conserve l'historique des chargements grâce à une colonne LOAD_DATE ajoutée à chaque table.

Le mécanisme anti-doublon dans l'ODS fonctionne ainsi :

- Un composant Lookup compare chaque ligne entrante de la DSA avec les lignes existantes dans l'ODS, en se basant sur la clé primaire (ex : TrackId pour Track).
- Si la ligne n'existe pas dans l'ODS (sortie No Match du Lookup), elle est insérée via un OLE DB Destination (INSERT).
- Si la ligne existe déjà (sortie Match du Lookup), une OLE DB Command exécute un UPDATE pour mettre à jour les colonnes qui ont pu changer (notamment le prix UnitPrice). Cette étape est cruciale pour que les changements de prix se propagent jusqu'au DWH et déclenchent le SCD Type 2.

Ce mécanisme est essentiel car il garantit que les modifications dans la source (comme un changement de prix) sont propagées à travers toutes les couches sans perdre la traçabilité.

C. Data Warehouse (DWH) et modèle en étoile

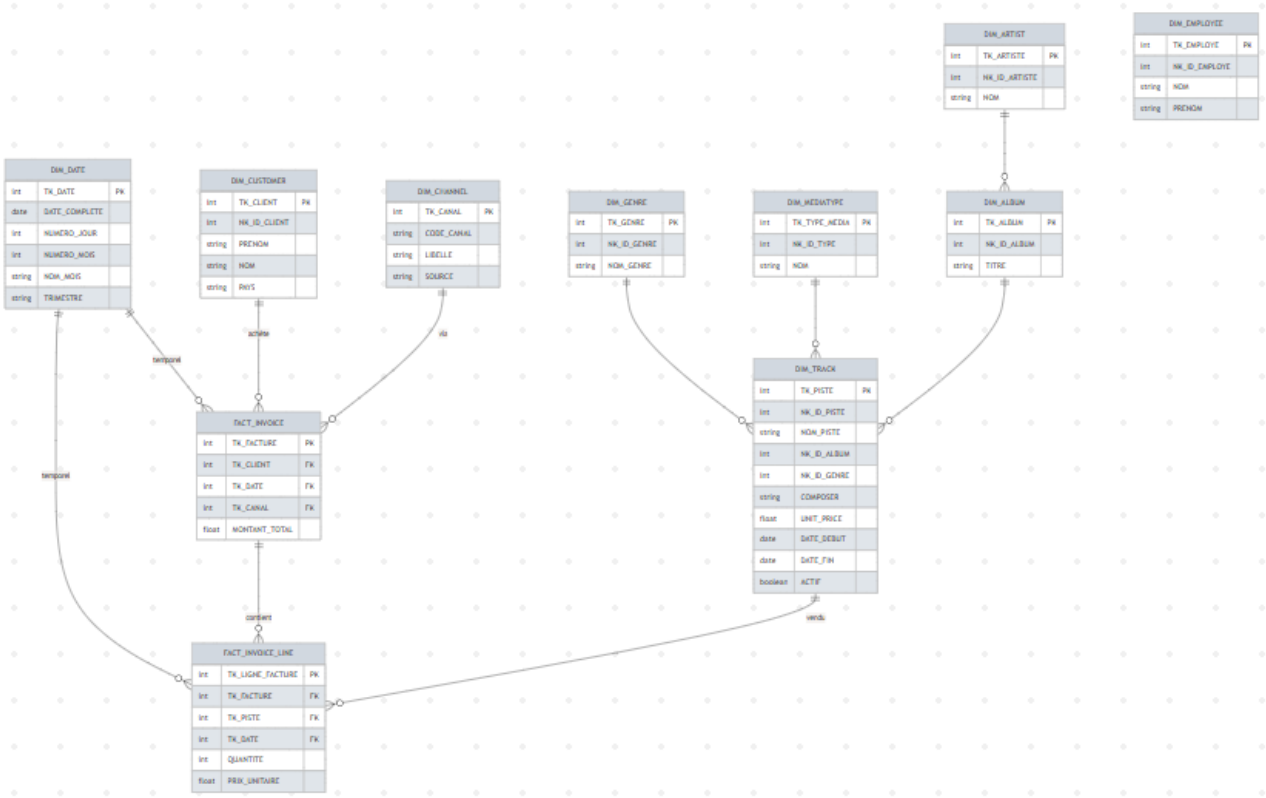
Le DWH est la couche finale, conçue pour l'analyse décisionnelle. Il utilise un modèle en étoile (star schema) où les tables de faits sont au centre, reliées aux tables de dimensions par des clés techniques (TK). Ce modèle est optimisé pour les requêtes analytiques (GROUP BY, SUM, COUNT) car il évite les jointures complexes du modèle transactionnel.

Le DWH comprend 9 tables de dimensions et 2 tables de faits. Les dimensions décrivent les axes d'analyse (Qui ? Quoi ? Quand ? Où ? Acheté Par Qui ? Vendu Par Qui ?) tandis que les faits contiennent les mesures (quantité, prix, montant total).

Table	Type	Description	Lignes
DIM_DATE	Dimension	Calendrier (CTE récursive)	~1884
DIM_TRACK	Dim. SCD2	Pistes + historique prix	3503+
DIM_CUSTOMER	Dimension	Clients (Online)	~59
DIM_CHANNEL	Dimension	Canal : Online / Store	2
DIM_GENRE	Dimension	Genres musicaux	~25
DIM_ARTIST	Dimension	Artistes	~275
DIM_ALBUM	Dimension	Albums	~347
DIM_EMPLOYEE	Dimension	Employés support	~8
DIM_MEDIATYPE	Dimension	Types de média	~5
FACT_INVOICE	Fait	Factures (Online & Store)	~1412
FACT_INVOICE_LINE	Fait	Lignes de facture	~4757

La particularité de DIM_DATE est qu'elle est générée entièrement par un script SQL utilisant une CTE récursive (Common Table Expression) et un MERGE. Ce script génère automatiquement toutes les dates entre le 01/01/2009 et le 31/12/2013, avec pour chaque date : le numéro du jour, le mois, le nom du mois, le trimestre et l'année.

Modèle en étoile du Data Warehouse



Bleu = Tables de faits | Gris = Tables de dimensions | SCD2 = Slowly Changing Dimension Type 2

Figure 3 : Modèle en étoile DWH_COMMON_LOKOUN_THYS

Dans ce schéma en étoile, les tables de faits (en bleu) occupent le centre. FACT_INVOICE_LINE contient une ligne par piste vendue, avec les clés techniques vers les dimensions DATE, TRACK et FACTURE.

FACT_INVOICE contient une ligne par facture, avec les clés vers CLIENT, DATE et CHANNEL. La dimension DIM_TRACK est la seule à gérer le SCD Type 2 : elle peut contenir plusieurs versions d'une même piste si le prix a changé.

4. Création des flux de données avec SSIS

Les flux de données constituent le cœur technique du projet. Ils sont implémentés sous forme de packages SSIS dans Visual Studio 2022. Chaque package (.dtsx) est un fichier XML qui décrit visuellement les étapes de transfert et de transformation. SSIS distingue deux niveaux de conception :

Control Flow : définit l'ordre des tâches (séquentiel ou parallèle). Les composants typiques sont :

Execute SQL Task (exécuter une requête),

Data Flow Task (flux de données),

Execute Package Task (appeler un autre package).

Les flèches vertes indiquent le succès, les rouges l'échec.

Data Flow : définit les transformations ligne par ligne à l'intérieur d'un Data Flow Task.

Les composants typiques sont :

OLE DB Source (lecture),
Derived Column (calcul de colonnes),
Lookup (recherche),
Conditional Split (routage conditionnel),
OLE DB Destination (écriture),
OLE DB Command (UPDATE/DELETE),
Row Count (comptage), Union All (fusion).

A. Flux Sources vers DSA

Chaque package DSA suit le même pattern : dans le Control Flow, un Execute SQL Task exécute un TRUNCATE TABLE pour vider la table cible, suivi d'un Data Flow Task pour transférer les données. 13 packages ont été créés (11 pour Chinook Online + 2 pour Magasin Store).

Dans le Data Flow, les composants sont chainés dans cet ordre : OLE DB Source (lecture de la table source) -> Derived Column (ajout des colonnes techniques TrackDate et CHANNEL_ID) -> Row Count (comptage des lignes dans une variable) -> OLE DB Destination (écriture dans la table DSA). Le composant Derived Column utilise des expressions SSIS : GETDATE () pour la date et un entier constant (1 ou 2) pour le CHANNEL_ID.

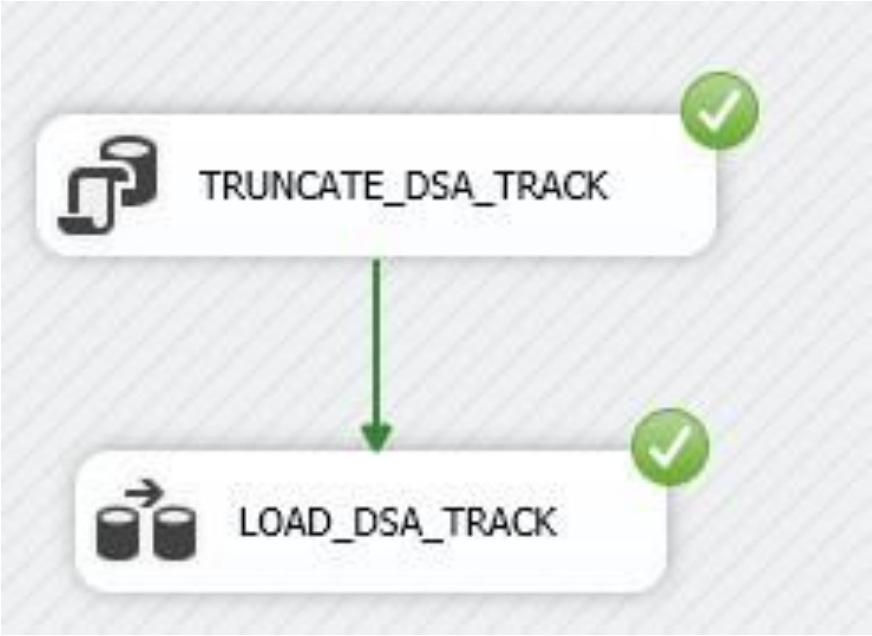


Figure 4 : Data Flow du package 01_LOAD_DSA_TRACK : OLE DB Source -> Derived Column -> Row Count -> OLE DB Destination

La Figure ci-dessus montre l'orchestrateur 00_RUN_ALL_DSA_LOADS.dtsx après une exécution réussie. Chaque boîte verte représente un Execute Package Task qui appelle un package DSA spécifique. Les packages sont exécutés séquentiellement de haut en bas.

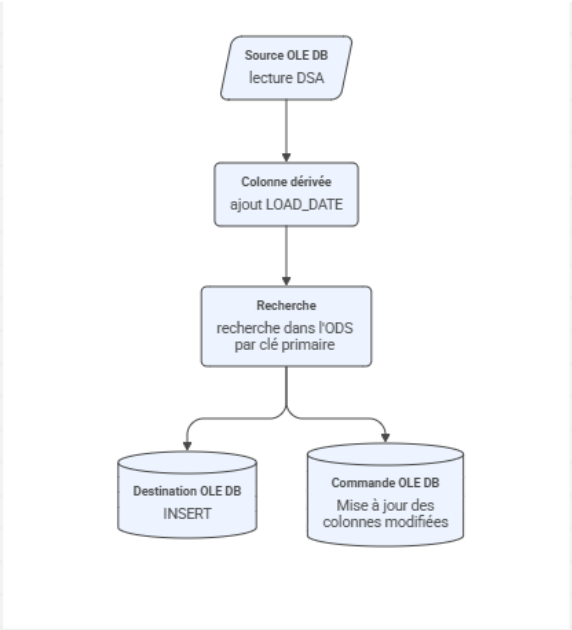
Figure 5 : Orchestration des 13 packages DSA via Execute Package Tasks (flèches vertes = succès)

Ce Data Flow illustre le pattern standard de tous les packages DSA. Les chiffres sur les flèches indiquent le nombre de lignes transférées (ici 3503 pistes).

B. Flux DSA vers ODS

Les flux DSA -> ODS sont plus complexes car ils doivent gérer le mécanisme anti-doublon décrit précédemment. Le pattern est le suivant : OLE DB Source (lecture DSA) -> Derived Column (ajout LOAD_DATE) -> Lookup (recherche dans l'ODS par clé primaire) -> deux sorties :

- Sortie No Match (nouvelle ligne) : OLE DB Destination pour INSERT.
- Sortie Match (ligne existante) : OLE DB Command pour UPDATE des colonnes modifiées (UnitPrice, etc.). C'est cette étape qui permet la propagation des changements de prix vers le DWH.



L'image ci-dessous montre le flux ODS pour la table Track. La sortie Match du Lookup alimente une OLE DB Command qui exécute un UPDATE SET UnitPrice = ? WHERE TrackId = ? pour mettre à jour le prix si celui-ci a changé dans la source. Sans cette étape, les modifications de prix ne seraient jamais propagées au DWH.

Figure 7: Data Flow de LOAD_ODS_TRACK: Lookup avec deux sorties (No Match = INSERT, Match = UPDATE via OLE DB Command)

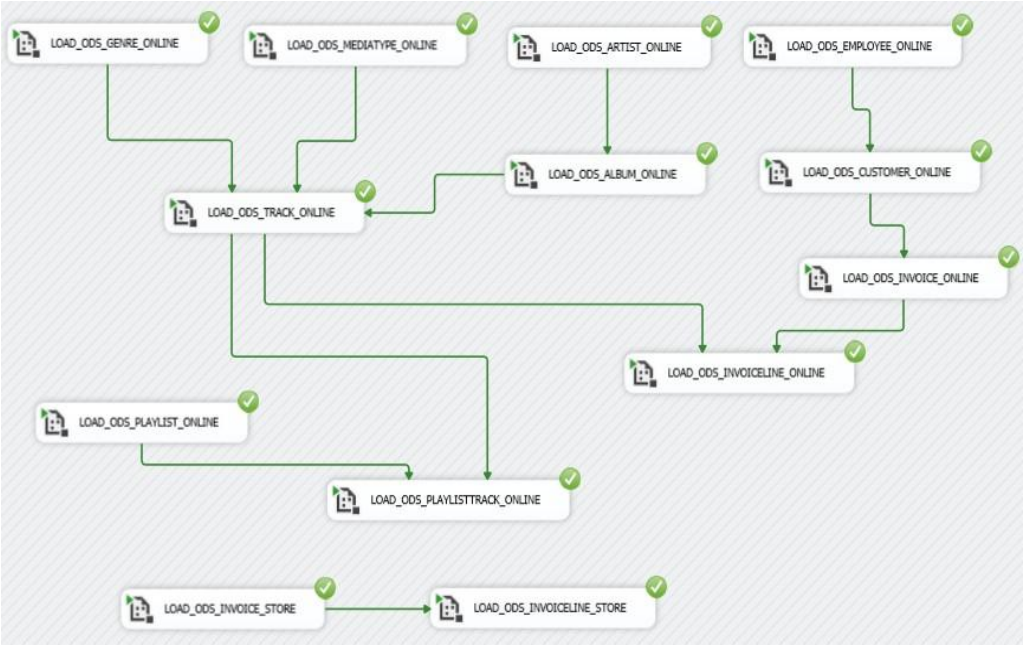


Figure 8 : Orchestration des packages ODS (exécution réussie)

C. Flux ODS vers DWH

Le chargement du DWH est le plus complexe car il implique la transformation du modèle transactionnel (tables normalisées) vers le modèle décisionnel (tables en étoile). Plusieurs types de chargements coexistent:

Dimensions simples : Lookup anti-doublon par clé naturelle (NK).
Si la dimension n'existe pas, INSERT avec génération automatique d'une clé technique (TK) via IDENTITY.
Exemples :DIM_GENRE, DIM_ARTIST, DIM_ALBUM, DIM_CUSTOMER.

DIM_DATE : générée par un Execute SQL Task contenant un script T-SQL avec une CTE récursive. Le MERGE insère les dates manquantes sans créer de doublons.

DIM_TRACK (SCD Type 2) : flux spécifique à 9 composants (détaillé dans la section suivante).

Tables de faits : TRUNCATE puis rechargement complet. Les données des deux sources (Online + Store) sont fusionnées via un composant Union All. Ensuite, des Lookups récupèrent les clés techniques (TK) des dimensions (DIM_DATE, DIM_TRACK, DIM_CUSTOMER, DIM_CHANNEL) avant l'insertion dans la table de faits.

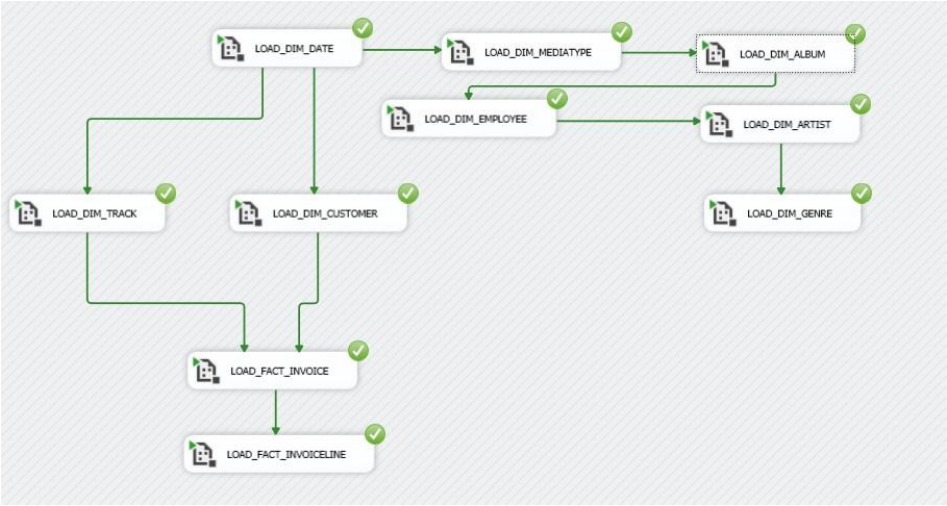


Figure 9 : Orchestration des 10 packages DWH : dimensions puis faits

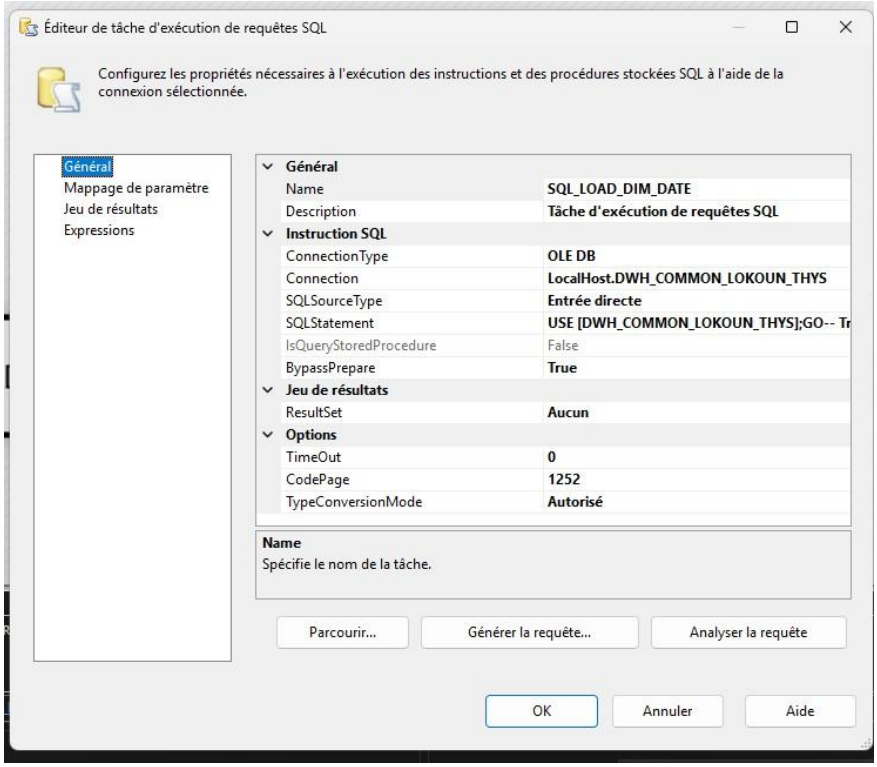


Figure 10 : Script SQL de DIM_DATE : CTE récursive générant les dates de 2009 à 2013 + MERGE anti-doublon

La CTE récursive commence au 01/01/2020 et ajoute un jour à chaque itération jusqu'au 31/12/2026. Le MERGE compare avec les dates déjà présentes (WHEN NOT MATCHED THEN INSERT) pour éviter les doublons lors des exécutions répétées. Chaque date est décomposée en numéro de jour, mois, nom du mois, trimestre et année.

Figure 11 : Exécution réussie du package DIM_DATE (1884 dates générées)

Figure 12 : FACT_INVOICE_LINE : Union All fusionnant Online et Store, puis Lookups TK vers les dimensions

Dans ce Data Flow, le composant Union All fusionne les lignes provenant de ODS_ONLINE et ODS_STORE en un seul flux. Ensuite, trois Lookups récupèrent les clés techniques : TK_FACTURE (via InvoiceId), TK_PISTE (via TrackId, ACTIF=1) et TK_DATE (via InvoiceDate). Le résultat est inséré dans FACT_INVOICE_LINE.

5. Gestion du SCD Type 2 sur DIM_TRACK

Le SCD Type 2 (Slowly Changing Dimension Type 2) est la fonctionnalité la plus complexe du projet. Son objectif est de conserver l'historique complet des changements de valeurs dans une dimension. Dans notre cas, nous suivons les changements de prix (UnitPrice) des pistes musicales dans DIM_TRACK.

Pourquoi le SCD Type 2 ?

Sans SCD2 (c'est-à-dire avec un simple UPDATE, appelé SCD Type 1), si le prix d'une piste passe de 0.99 à 0.49, l'ancien prix est écrasé et perdu définitivement. Impossible ensuite de savoir quel prix était en vigueur lors des ventes passées. Le SCD Type 2 résout ce problème en conservant les deux versions.

Critère	SCD Type 1 (UPDATE)	SCD Type 2 (notre choix)
Principe	Écrase l'ancienne valeur	Conserve toutes les versions
Historique	Perdu définitivement	Préservé intégralement
Colonnes suppl.	Aucune	DATE_DEBUT, DATE_FIN, ACTIF
Nb de lignes	Constant	Augmente (+1 par changement)
Analyse du CA	Anciens prix perdus	Comparaison ancien/nouveau CA
Complexité SSIS	1 simple UPDATE	Lookup + Split + Command (9 composants)
Cas d'usage	Corrections d'erreurs	Changements métier (prix, statuts)

Fonctionnement pas à pas

L'utilisation de l'Assistant SCD permet d'automatiser la logique complexe du SCD Type 2.

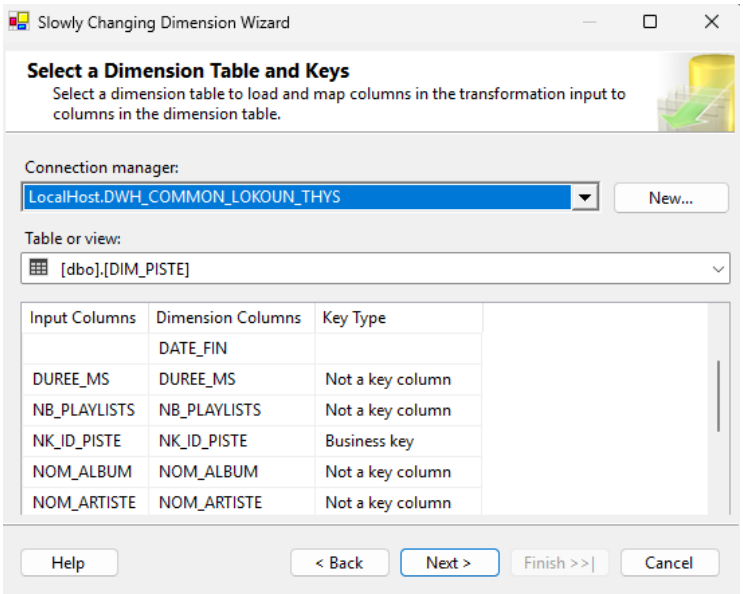
Prenons l'exemple d'une piste dont le prix passe de **0.99 €** à **0.49 €** :

Étape 1 : Lecture de la source ODS

Le flux commence par le composant **SRC_ODS_TRACK**. Il extrait les données les plus récentes de votre table ODS. À ce stade, le prix est déjà de 0.49 € à la suite des traitements précédents.

Étape 2 : Identification via la Business Key

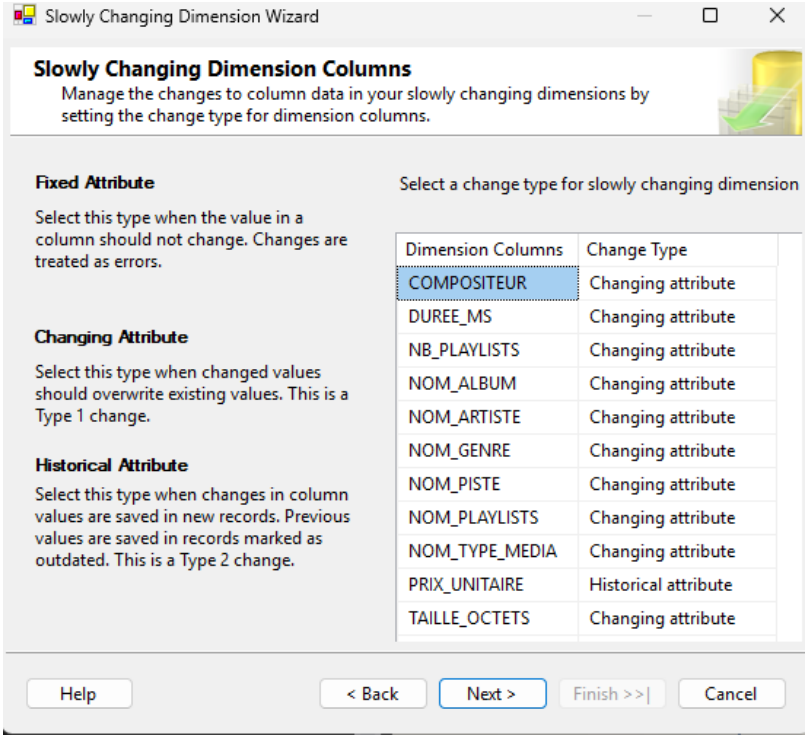
Dans l'assistant, vous avez défini **NK_ID_PISTE** comme **Business Key**. SSIS utilise cette clé pour comparer les données entrantes avec celles déjà présentes dans la dimension DIM_TRACK du Data Warehouse.



Étape 3 : Analyse des attributs (SCD1 vs SCD2)

C'est ici que l'assistant applique la logique que vous avez configurée :

- **Historical Attribute (SCD2) :** Vous avez marqué **UNIT_PRICE** comme attribut historique. SSIS détecte que 0.49 (ODS) ≠ 0.99 (DWH).
- **Changing Attribute (SCD1) :** Pour les colonnes comme NOM_ALBUM ou COMPOSER, SSIS mettra à jour toutes les lignes existantes si un changement est détecté, sans créer d'historique.



Étape 4 : Fermeture de l'ancienne version

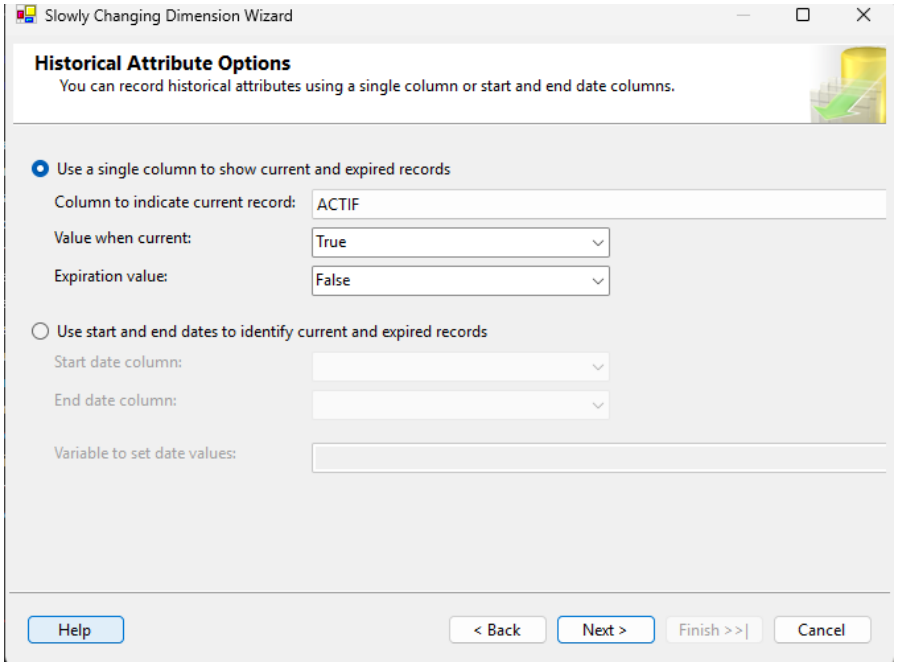
Comme le prix a changé, l'assistant dirige la ligne vers la sortie **Historical Attribute Output**.

- Il génère une **Commande OLE DB** qui exécute un UPDATE pour "fermer" la version actuelle.
- L'indicateur **ACTIF** passe de **True** à **False**.

Étape 5 : Insertion de la nouvelle version

Une fois l'ancienne ligne fermée, le flux insère une nouvelle ligne via la **Destination d'insertion** :

- Le nouveau prix est enregistré (0.49 €).
- L'indicateur **ACTIF** est mis à **True**.
- Les métadonnées temporelles (Date de début) sont automatiquement gérées pour marquer le début de cette nouvelle période de validité.



6. Validation et vérification des données

La validation est une étape essentielle de tout projet d'intégration. Elle permet de vérifier que les données n'ont pas été altérées, perdues ou dupliquées au cours du processus ETL. Nous avons mis en place trois types de contrôles :

A. Test du SCD Type 2

Pour tester le SCD2, nous avons modifié volontairement le prix de 48 pistes dans la base source Chinook (UnitPrice de 0.99 à 0.49) via un UPDATE ciblé. Puis nous avons re-exécuté l'ETL complet (00_RUN_ALL). Le résultat attendu : dans DIM_TRACK, chaque piste modifiée doit apparaître en deux versions.

La requête de vérification utilisée :

```
SELECT CASE WHEN ACTIF = 0 THEN 'AVANT (fermée)'
ELSE 'APRES (active)' END AS [VERSION],
    NK_ID_PISTE, NOM_PISTE, UNIT_PRICE,
    DATE_DEBUT, DATE_FIN, ACTIF
FROM [DWH_COMMON_LOKOUN_THYS].[dbo].[DIM_TRACK]
WHERE NK_ID_PISTE BETWEEN 2358 AND 2370
ORDER BY NK_ID_PISTE, ACTIF;
```

	TK_PISTE	NK_ID_PISTE	NOM_PISTE	UNIT_PRICE	DATE_DEBUT	DATE_FIN	ACTIF
1	2358	2358	The Power Of Equality	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
2	3504	2358	The Power Of Equality	0.49	2026-03-01 22:29:39.6430000	NULL	1
3	2359	2359	If You Have To Ask	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
4	3505	2359	If You Have To Ask	0.49	2026-03-01 22:29:39.6430000	NULL	1
5	2360	2360	Breaking The Girl	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
6	3506	2360	Breaking The Girl	0.49	2026-03-01 22:29:39.6430000	NULL	1
7	2361	2361	Funky Monks	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
8	3507	2361	Funky Monks	0.49	2026-03-01 22:29:39.6430000	NULL	1
9	2362	2362	Suck My Kiss	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
10	3508	2362	Suck My Kiss	0.49	2026-03-01 22:29:39.6430000	NULL	1
11	2363	2363	I Could Have Lied	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
12	3509	2363	I Could Have Lied	0.49	2026-03-01 22:29:39.6430000	NULL	1
13	2364	2364	Mellowship Slinky In B Major	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
14	3510	2364	Mellowship Slinky In B Major	0.49	2026-03-01 22:29:39.6430000	NULL	1
15	2365	2365	The Righteous & The Wicked	0.99	2026-03-01 10:43:50.3840000	2026-02-28 22:29:39.5570000	0
16	3511	2365	The Righteous & The Wicked	0.49	2026-03-01 22:29:39.6430000	NULL	1

Figure 15 : Résultat SCD2 dans SSMS : chaque piste apparaît deux fois (AVANT à 0.99, APRES à 0.49)

L'image confirme le bon fonctionnement : pour chaque piste modifiée, la version AVANT (ACTIF=0) conserve le prix original 0.99 avec une DATE_FIN correspondant au moment du changement. La version APRES (ACTIF=1) porte le nouveau prix 0.49 avec une DATE_DEBUT correspondante et DATE_FIN = NULL.

B. Comptage des lignes par couche

Le nombre de lignes doit rester constant à travers les couches (Source -> DSA -> ODS -> DWH) pour garantir qu'aucune donnée n'est perdue. L'exception est DIM_TRACK qui augmente grâce au SCD2 : 3503 lignes de base + 48 versions historisées = 3551 lignes au total (dont 3503 actives et 48 fermées).

TABLE	SOURCE	DSA	ODS	DWH
--- CHINOOK (Online) ---				
Track	3503	3503	3503	3503 actifs
Album	347	347	347	-
Artist	275	275	275	-
Customer	59	59	59	-
Genre	25	25	25	-
Invoice (Online)	412	412	412	-
InvoiceLine (Online)	2240	2240	2240	-
--- MAGASIN (Store) ---				
Invoice (Store)	1000	1000	1000	-
InvoiceLine (Store)	4438	4438	4438	-
--- DWH (fusion) ---				
DIM_DATE	-	-	-	1884
DIM_TRACK (total)	-	-	-	3551
dont ACTIF = 1	-	-	-	3503
dont ACTIF = 0 (historique SCD2)	-	-	-	48
DIM_CHANNEL	-	-	-	2
FACT_INVOICE (Online+Store)	-	-	-	7060
FACT_INVOICE_LINE (Online+Store)	-	-	-	26712

Figure 16 : Suivi des lignes par couche : le nombre reste constant de la source au DWH (cohérence vérifiée)

Ce tableau montre que les comptages sont identiques à chaque couche pour toutes les tables (Track, Album, Artist, Customer, Genre, Invoice, InvoiceLine). Seule DIM_TRACK dans le DWH affiche un nombre supérieur, ce qui est le comportement attendu du SCD2.

C. Évolution du chiffre d'affaires (CA)

Le CA (Quantité x Prix Unitaire) est calculé à chaque couche et pour chaque canal (Online et Store) séparément. L'objectif est de vérifier qu'il n'y a aucune perte de données monétaires au cours du processus ETL.

Le CA doit rester identique de la source au DWH. Dans le DWH, deux valeurs de CA sont calculées : le CA avec les prix actuels (ACTIF=1) et le CA avec les prix d'origine (ACTIF=0). La différence entre les deux s'explique exclusivement par le SCD2 : les 48 pistes dont le prix est passé de 0.99 à 0.49 génèrent un CA inférieur avec les prix actuels.

	CANAL	COUCHE	NB_LIGNES	CA
1	ONLINE	1-SOURCE	2240	2328.60
2	ONLINE	2-DSA	2240	2328.60
3	ONLINE	3-ODS	2240	2328.60
4	ONLINE	4-DWH	2240	2328.60

	CANAL	COUCHE	NB_LIGNES	CA
1	STORE	1-SOURCE	4438	4606.62
2	STORE	2-DSA	4438	4606.62
3	STORE	3-ODS	4438	4606.62
4	STORE	4-DWH	4438	4606.62

Figure 17 : Évolution du CA par couche et par canal : zéro perte de la source au DWH

L'image confirme : le CA Source = CA DSA = CA ODS = CA DWH (prix historiques). Le CA DWH avec prix actuels est légèrement inférieur, ce qui est cohérent avec la baisse de prix de 48 pistes. Cette différence constitue la preuve que le SCD2 fonctionne correctement et permet de comparer l'évolution du CA dans le temps.

IV. Automatisation de l'ETL

En production, un ETL ne peut pas être exécuté manuellement à chaque fois. SQL Server propose deux mécanismes d'automatisation : SQL Server Agent pour la planification, et les triggers T-SQL pour la détection événementielle.

1. SQL Server Agent

SQL Server Agent est un service intégré à SQL Server qui permet de planifier l'exécution automatique de tâches. Un Job se compose de Steps (les étapes à exécuter) et de Schedules (les horaires d'exécution). Dans notre cas, le Job exécuterait le package 00_RUN_ALL.dtsx chaque nuit :

```
  USE msdb;
  EXEC  sp_add_job  @job_name  =  'ETL_RUN_ALL_NIGHTLY';      EXEC  sp_add_jobstep  @job_name  =
'ETL_RUN_ALL_NIGHTLY',       @step_name = 'Executer 00_RUN_ALL',
      @subsystem = 'SSIS',
      @command = '\SSISDB\ETL_Chinook\00_RUN_ALL.dtsx';
  EXEC sp_add_schedule @schedule_name = 'Nuit_2h',
      @freq_type = 4, @active_start_time = 020000;
```

Le paramètre freq_type = 4 signifie "tous les jours", et active_start_time = 020000 signifie "02h00". Ainsi, l'ETL complet serait exécuté automatiquement chaque nuit à 2h du matin, moment où la charge sur le serveur est minimale.

2. Triggers T-SQL

En complément de la planification régulière, un trigger T-SQL peut détecter automatiquement les modifications de prix dans la source et lancer l'ETL immédiatement. Cela permet une réactivité maximale : dès qu'un prix change dans Chinook, le SCD2 est déclenché.

```
  CREATE  TRIGGER  trg_Track_PriceChange
ON   [Chinook_LOKOUN_THYS].[dbo].[Track]
AFTER UPDATE AS
  BEGIN
      IF UPDATE(UnitPrice)
          EXEC             msdb.dbo.sp_start_job
@job_name = 'ETL_RUN_ALL_NIGHTLY';
  END;
```

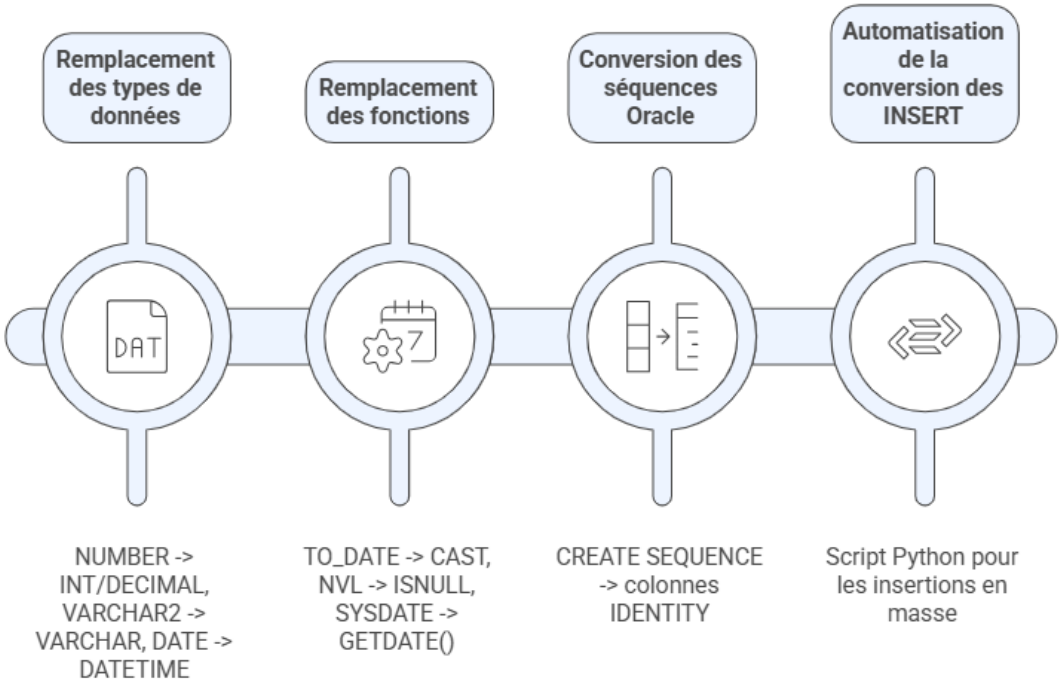
Le trigger AFTER UPDATE se déclenche après chaque UPDATE sur la table Track. La condition IF UPDATE(UnitPrice) vérifie que c'est bien la colonne UnitPrice qui a été modifiée. Si oui, il lance le Job ETL via sp_start_job, ce qui déclenche toute la chaîne DSA -> ODS -> DWH et met à jour le SCD2 dans DIM_TRACK.

V. Difficultés rencontrées et solutions

Tout au long du projet, nous avons rencontré plusieurs difficultés techniques qui ont nécessité des recherches et des adaptations. Ces difficultés font partie intégrante de l'apprentissage et ont renforcé notre compréhension des mécanismes ETL.

1. Conversion Oracle vers SQL Server

La base Magasin était fournie en syntaxe Oracle (PL/SQL). La conversion vers T-SQL a nécessité : le remplacement des types de données (NUMBER -> INT/DECIMAL, VARCHAR2 -> VARCHAR, DATE -> DATETIME), le remplacement des fonctions (TO_DATE -> CAST, NVL -> ISNULL, SYSDATE -> GETDATE()), et la conversion des séquences Oracle (CREATE SEQUENCE) en colonnes IDENTITY. Un script Python a été utilisé pour automatiser la conversion des INSERT en masse.



2. SCD2 ne se déclenchait pas

Après modification des prix dans Chinook et réexécution de l'ETL, DIM_TRACK ne montrait aucune ligne historisée. Le problème était dans le flux ODS : le Lookup n'insérait que les nouvelles lignes (No Match) mais ignorait les lignes existantes (Match). Le prix modifié dans la source ne se propageait donc jamais jusqu'à l'ODS, et donc jamais jusqu'au DWH. Solution : ajout d'un OLE DB Command sur la sortie Match du Lookup ODS pour exécuter un UPDATE. Important : nous ne pouvions pas TRUNCATE l'ODS car cela aurait détruit la traçabilité historique (LOAD_DATE).

3. Doublons dans les tables de faits

Après plusieurs exécutions de l'ETL, FACT_INVOICE_LINE contenait 5 fois trop de lignes. Cause : les faits étaient rechargés sans vider la table, créant des doublons à chaque exécution. Solution : ajout d'un Execute SQL Task avec TRUNCATE TABLE au début des packages de chargement des faits. Contrairement aux dimensions (qui nécessitent un Lookup anti-doublon), les faits sont rechargés intégralement (Full Reload).

4. Noms de colonnes incohérents entre couches

Les colonnes portent des noms différents selon la couche : UnitPrice dans Chinook/DSA/ODS vs PRIX_UNITAIRE dans le DWH, TrackId vs NK_ID_PISTE, etc. Cette incohérence a causé des erreurs dans les requêtes de vérification du CA. Solution : utilisation systématique des noms DWH pour les requêtes analytiques, et documentation précise des correspondances entre colonnes.

5. Erreur de connexion SQL Server

L'erreur 15404 ("Impossible d'obtenir des informations sur l'utilisateur Windows NT") apparaissait lors de certaines opérations. Cause : le propriétaire de la base était un compte Windows non reconnu. Solution : ALTER AUTHORIZATION ON DATABASE::[nom_base] TO [sa] pour transférer la propriété au compte sa (System Administrator).

VI. Conclusion

Ce projet nous a permis de construire une chaîne ETL complète avec SQL Server et SSIS, en partant de deux sources hétérogènes (Chinook et Magasin) pour aboutir à un Data Warehouse unifié. L'ensemble des objectifs fixés ont été atteints :

- 7 bases de données créées et alimentées (2 sources + 2 DSA + 2 ODS + 1 DWH)
- Plus de 30 packages SSIS orchestrés hiérarchiquement (RUN_ALL -> DSA/ODS/DWH -> packages individuels)
- SCD Type 2 fonctionnel et vérifié avec 48 changements de prix (2 versions par piste modifiée)
- Fusion réussie des canaux Online et Store dans le DWH avec traçabilité via CHANNEL_ID
- Zéro perte de données vérifiée par comptage de lignes et calcul du CA à chaque couche
- Automatisation possible via SQL Server Agent et triggers T-SQL

Par rapport au semestre 3, ce projet représente une évolution majeure sur trois plans : la maîtrise d'un second écosystème ETL (Microsoft vs Oracle), la gestion de deux sources de données hétérogènes, et l'implémentation effective du SCD Type 2 que nous n'avions pas réussi à finaliser au S3. La compréhension des mécanismes ETL et de la modélisation décisionnelle est désormais solide.

Parmi les améliorations envisageables :

- Automatisation complète via SQL Server Agent avec alertes par email en cas d'échec
- Ajout de contrôles de qualité plus fins (Data Profiling, assertions sur les volumes)
- Intégration de nouvelles sources (API, fichiers CSV, bases NoSQL)
- Séparation de Invoice en dimension dédiée pour optimiser les performances des requêtes SELECT

Ces améliorations représenteraient environ 10 à 15 heures de travail supplémentaires. Le système actuel répond pleinement aux objectifs fixés.

VII. Bilan personnel

Kris LOKOUN

-> Compétences ETL approfondies : Oracle/ODI au S3, SQL Server/SSIS au S4

-> Implémentation manuelle du SCD Type 2 : logique d'historisation complète

-> Débogage avancé : traçage des données couche par couche pour localiser les problèmes

-> Fusion de deux sources hétérogènes : défis réels de l'intégration multi-sources

-> Maîtrise des outils professionnels de la BI (VS, SSMS, SQL Server Agent)

-> Cette expérience me prépare aux projets futurs de gestion de données à grande échelle

Abel THYS

-> Compétences T-SQL améliorées : MERGE, CTE récursives, triggers, IDENTITY

-> Conversion de syntaxe Oracle vers SQL Server : compréhension des différences entre SGBD

-> Compréhension des clés techniques (TK) et naturelles (NK) dans un DWH

-> Travail en binôme : collaboration efficace et répartition équilibrée des tâches

-> Capacité à déboguer des problèmes complexes en équipe

Apprentissages communs

- Maîtrise de deux écosystèmes ETL complets (Oracle+ODI et Microsoft+SSIS)
- Architecture DSA -> ODS -> DWH maîtrisée de bout en bout
- Gestion de l'historique avec le SCD Type 2 (théorie et pratique)
- Importance de la vérification systématique à chaque étape de l'ETL
- Autonomie accrue : installation et configuration des outils sans dépendance à l'IUT

VIII. Remerciements

Nous tenons à remercier sincèrement Monsieur Yannick Le Véler pour son accompagnement tout au long de ce projet. Ses conseils techniques, sa pédagogie et sa disponibilité nous ont permis de progresser efficacement et de mener ce projet à bien. Grâce à ses précieux retours, nous avons pu surmonter les difficultés rencontrées et approfondir notre compréhension de l'intégration de données.

Nous remercions également l'Université de Lille et l'IUT de Roubaix pour les ressources matérielles et logicielles mises à notre disposition, qui ont rendu possible la réalisation de ce projet dans de bonnes conditions.